QA Strategy and Execution Document

Project: MindBridge — Shareable Content Feature

Your name: Tony

Date: October 30, 2027

Jira project name: Mindbridge QA Testing

Complete all four sections below. Use the section structure exactly as provided — the graded assignment that follows this lab looks for information in these specific sections. When finished, export this document as a single PDF for submission.

Section 1: Architecture and RACI Matrix

For each application layer, assign a role to the QA Team and the Dev Team. Use one of: **R** (Responsible), **A** (Accountable), **C** (Consulted), or **I** (Informed). Do not leave any cell blank.

Layer	QA Team Role	Dev Team Role
UI	R	A
API	R	A
DB	R	A

Section 2: STLC Strategy and Environments

In a few bullet points, explain which techniques you will use and where you will test. Your strategy must name **at least one static technique**, **at least one dynamic technique**, and **the test environment**.

Static testing technique(s):

• Review the user story, acceptance criteria and Definition of Done (DOD) to identify missing requirements, ambiguities, and potential defects before development and execution.

• **Requirements and Design Review**: Review the requirements and design documentation to determine whether the share link is intended to support multiple recipients or a single recipient, ensuring the expected behaviour is clearly defined.

Code Review: Review the source code (if available) to identify potential defects in how the share link handles multiple recipients, including any limitations or inconsistencies with the requirements.

Dynamic testing technique(s):

• Functional Testing: Execute the share link functionality to verify that users can successfully generate and share a link with the intended recipients.

• **Multiple Recipient Testing:** Test sharing the link with multiple recipients to verify whether all intended recipients can access the shared content successfully.

Positive and Negative Testing: Test valid(happy path) and invalid scenarios, such as sharing a link with valid recipients, entering invalid email addresses, leaving recipient fields empty, or attempting to access an invalid or expired share link.

Boundary Value Analysis: Test the maximum number of recipients allowed (if there is a defined limit), including testing at, below, and above the limit to identify potential defects.

Compatibility Testing: Verify that the share link works across different browsers, devices, and operating systems.

Access Control Testing: Verify that only intended recipients can access the shared content & that no unauthorised users cannot access it.

Test environment:

1. Test Application: Access to the deployed application in the QA/test environment, including the share link functionality.

2. Test Accounts: Multiple test user accounts to verify sharing functionality between different users and recipients.

3. Browsers: Google Chrome, Microsoft Edge, Safari, and Firefox (where applicable) to verify browser compatibility.

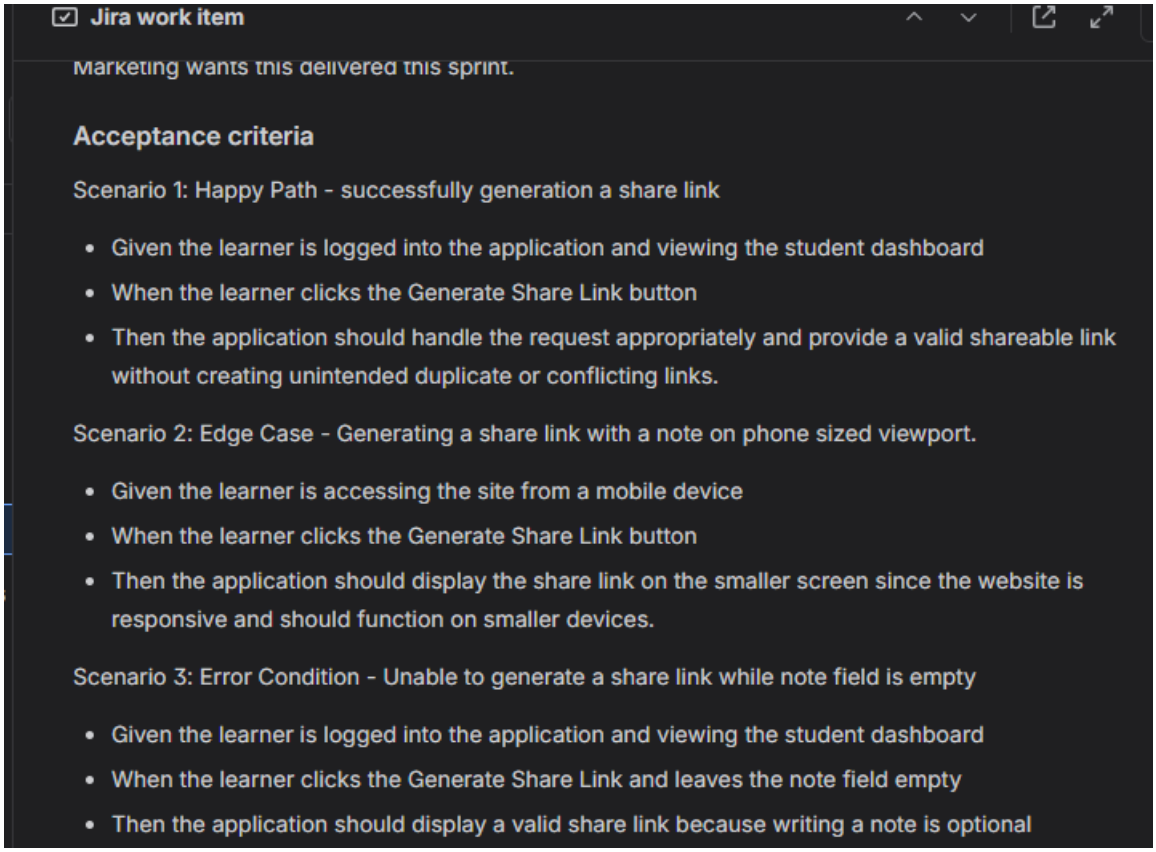
4. Devices: Desktop, laptop, tablet, and mobile devices to verify the functionality across different screen sizes and platforms. Emulators can also be used instead of physical devices.

5. Test Data: Valid and invalid email addresses, multiple recipient details, and test content to verify different sharing scenarios.

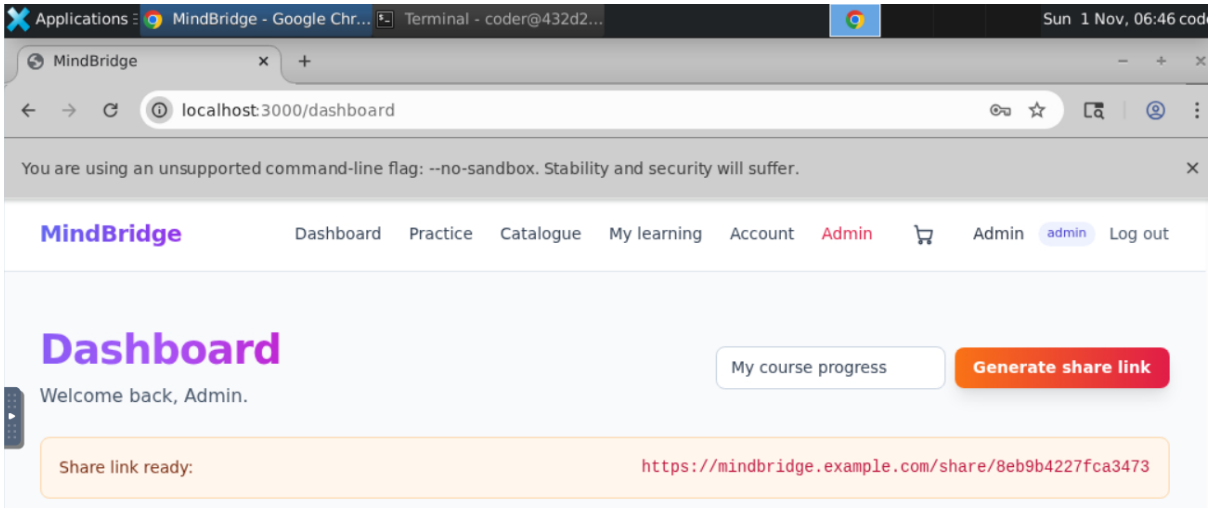
6. Staging Environment

Section 3: BDD Acceptance Criteria

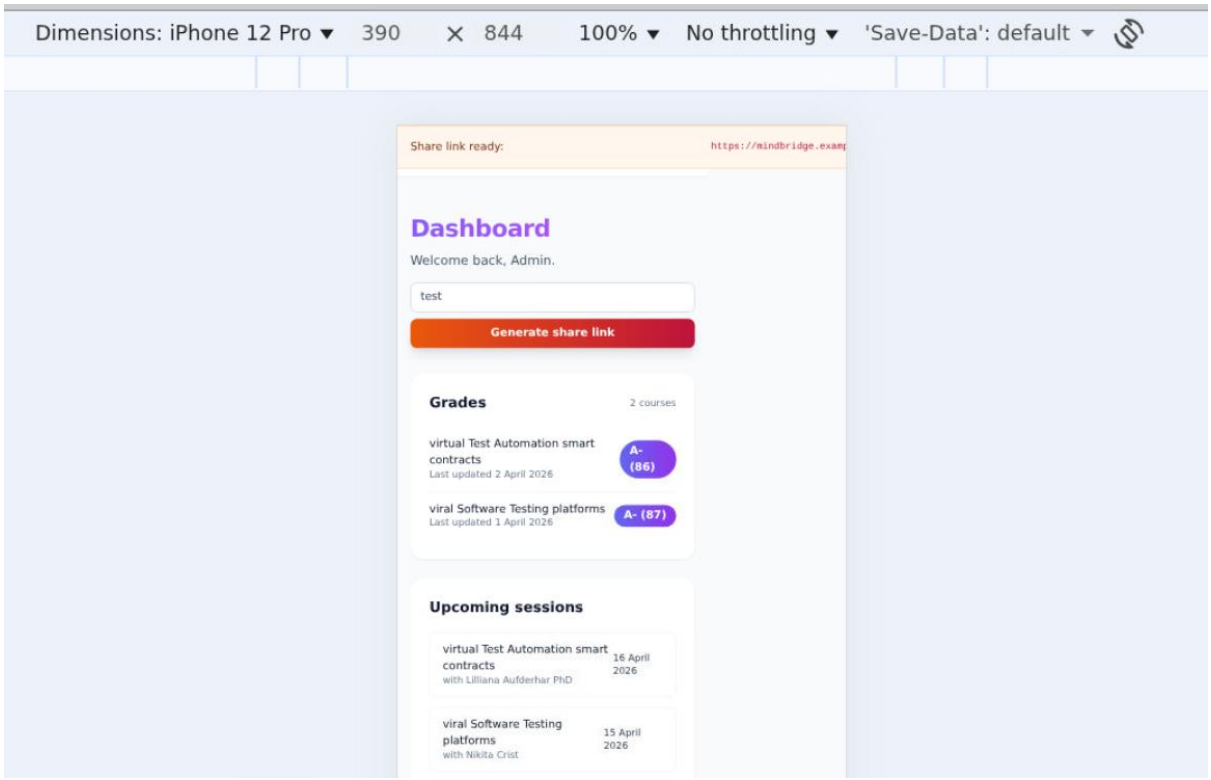
After editing the Jira user story to add your three Given-When-Then scenarios (a happy path, an edge case, and an error condition), take a screenshot of the updated ticket and insert it below. All three scenarios must be clearly visible in the description field.



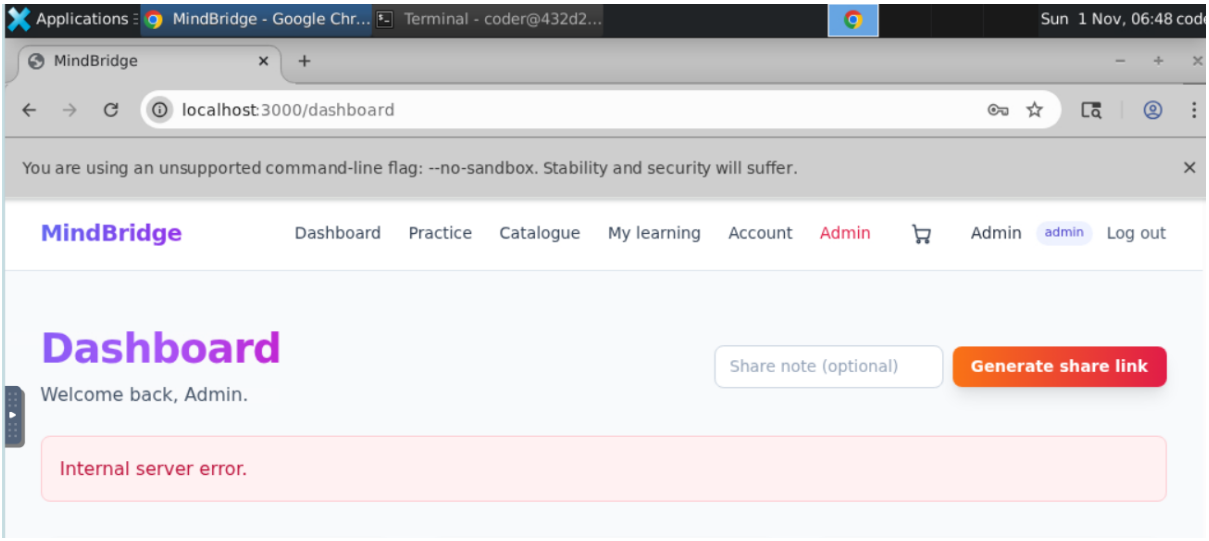
Scenario 1: Happy path: Type a short note (for example, my course progress) and select **Generate share link**. A "Share link ready" banner with a URL should appear.



Scenario 2: Edge Case : Generating a share link with a note on phone sized viewport.



Scenario 3: Error condition: Clear the note field, leave it empty, and select **Generate share link** again.



Section 4: Defect Log and Execution Evidence

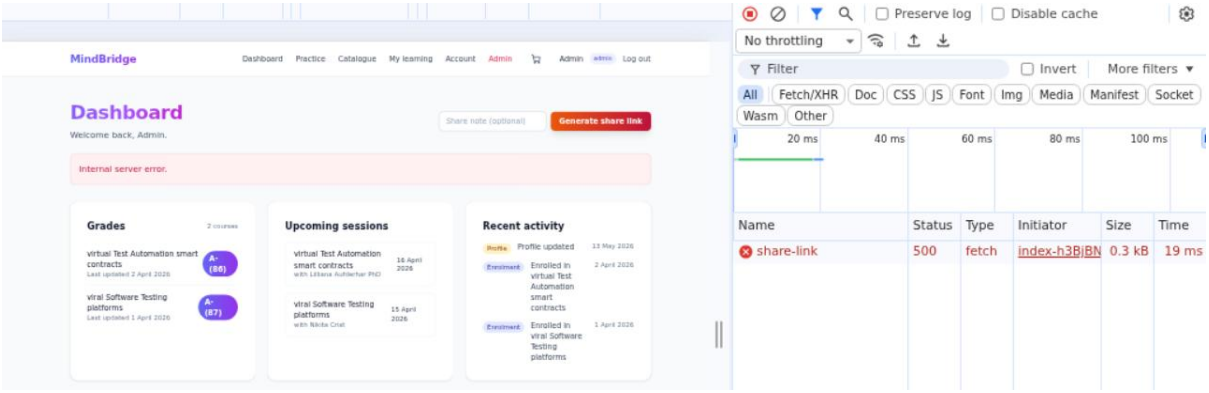
Document the seeded defect you found while executing your BDD scenarios against the staging environment.

Defect title	Description	Steps to reproduce	Expected result	Actual result	Severity
API Defect	something goes wrong when the application communicates with the server. Keep DevTools open on the Network tab while you submit the empty-note request and watch the share-link request's status code.	1.Clear the note field 2.Leave it empty 3.Select generate share link again	Note is optional, not required. Meaning if users leave it empty there should be no error.	Internal serve error message received & link is not generated.	Severe
UI Defect	Visual layout breaks under certain conditions. Specifically under phone sized viewport(EX Iphone 12 Pro)	1. Open Emulator 2. Set to iPhone 12 pro or another mobile device. 3. Generate a share link with note	It should generate a share link & be visible due to a responsive design.	Share link is generate but not visible on mobile devices due to the webpage not being responsive to different sized screens.	

Insert a screenshot that clearly shows evidence of the defect (any one of the defects):

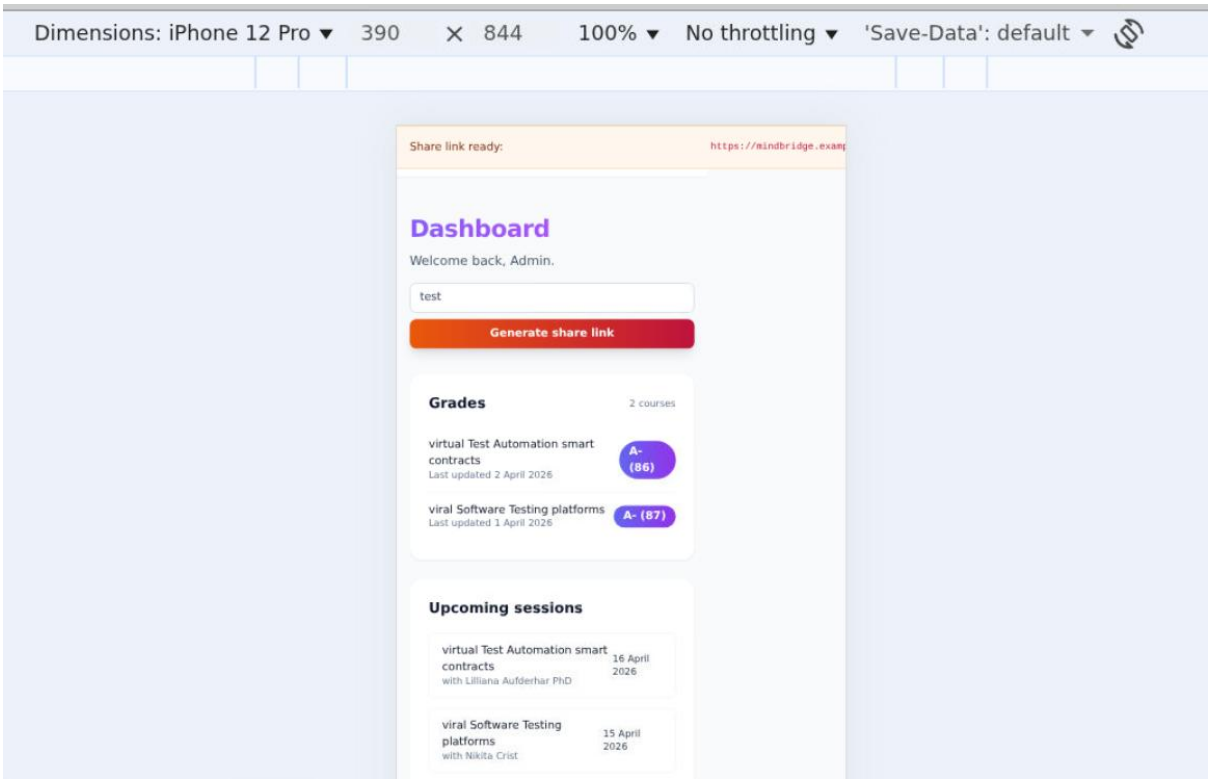
[Insert your defect evidence screenshot here]

- **API Defect**—something goes wrong when the application communicates with the server. Keep DevTools open on the Network tab while you submit the empty-note request and watch the share-link request's status code.



UI Defect—something in the visual layout breaks under certain conditions

Edge case: Open DevTools (F12), enable the device toolbar (Ctrl+Shift+M), pick a phone-sized viewport (for example, iPhone SE), and generate a share link with a note.



One decision I made during the lab that improved the quality of my QA Strategy document was capturing multiple screenshots of the bugs and errors I discovered during testing and including them in my defect documentation. I made this decision because I wanted to provide clear visual evidence of the issues rather than just describing them in writing. This improved the quality of my document by making the defects easier to understand, improving traceability, and providing developers with evidence they could use to investigate and reproduce the problems. A weaker version of this decision would have been simply writing a brief description of each error without attaching screenshots or including the relevant error messages and application screens. This would have made it harder for developers to understand exactly what went wrong and investigate the defects.